

// LIVING DOCUMENTATION FOR AI-BUILT SOFTWARE REV. 2026-07-04
STATUS: TEMPLATE

The docs/ Playbook for Self- Documenting Repositories

A structured system for turning a codebase — especially one built quickly with AI — into a repository any developer or AI agent can understand, trust, and safely extend without reverse-engineering the source.

PURPOSE Eliminate
cognitive debt

AUDIENCE Humans +
AI agents

FORMAT 26-file
docs/ tree

PRINCIPLE Explain why
not just what

Table of Contents

PART I – FOUNDATIONS

1. Why This Exists — Cognitive Debt & the Case for docs/	03
2. The docs/ Folder Structure	04
3. How to Use This Playbook	05

PART II – CORE REFERENCE DOCUMENTS

4. PROJECT_OVERVIEW.md	06
5. ARCHITECTURE.md & Diagrams	07
6. FOLDER_STRUCTURE.md & TECH_STACK.md	09
7. DATABASE.md	10
8. API_REFERENCE.md	11
9. AUTHENTICATION.md	13
10. STATE_MANAGEMENT.md & ROUTES.md	14
11. COMPONENTS.md	15

PART III – FEATURE DOCUMENTATION

12. FEATURES/ — Per-Feature Template	16
13. Worked Example — login.md	17

PART IV – GOVERNANCE & AI SAFETY

14. DESIGN_DECISIONS.md	18
15. AI_CONTEXT.md — Rules for Future AI Agents	19

16. SECURITY.md & KNOWN_LIMITATIONS.md	21
--	----

PART V – OPERATIONS

17. DEPLOYMENT.md & TESTING.md	22
--------------------------------	----

18. PERFORMANCE.md & TROUBLESHOOTING.md	23
---	----

19. CHANGELOG.md & FUTURE_IMPROVEMENTS.md	24
---	----

20. Documentation Quality Checklist	25
-------------------------------------	----

Why This Exists

docs/README.md

Software built quickly — especially with AI assistance — accumulates **cognitive debt**: the gap between what a codebase does and what anyone (human or AI) currently understands about it. Unlike technical debt, cognitive debt doesn't show up in a linter. It shows up six months later, when a new contributor — or a new AI session with no memory of the old one — has to rediscover, from scratch, every decision that was once obvious.

WHY IT MATTERS

Every AI coding session starts from zero. Without a written record of intent, an agent can only infer "why" from "what" — and inference drifts. A login form's odd validation order might be a deliberate fraud-prevention measure, or it might be an accident nobody caught. Without docs, both look identical.

This playbook defines a `docs/` tree that becomes the **single source of truth** for a repository: not a copy of the code in prose, but the reasoning layer that sits above it. The goal is that someone unfamiliar with the repository can understand, debug, and extend it confidently by reading `docs/` alone.

The three failure modes this system prevents

Failure mode	What it looks like	How docs/ prevents it
Re-explaining the codebase every session	Every new chat starts with "let me explore the repo first"	<code>AI_CONTEXT.md</code> gives an agent instant orientation
Silent regressions from AI edits	An agent "fixes" a deliberate workaround, breaking a case it didn't know about	<code>KNOWN_LIMITATIONS.md</code> and risk flags in <code>AI_CONTEXT.md</code>
Architectural drift	Two features solve the same problem two different ways	<code>DESIGN_DECISIONS.md</code> records the reasoning once, for reuse

PRINCIPLE

Document *why*, not just *what*. The code already says what it does. Documentation earns its keep only when it explains the reasoning the code cannot express — trade-offs, constraints, and the alternatives that were rejected.

The docs/ Folder Structure

Every file below has one job. None of them should overlap — if two files start repeating each other, that's a sign one of them has drifted from its purpose.

```
docs/
├─ README.md                entry point, links everything
├─ PROJECT_OVERVIEW.md      beginner-friendly explanation
├─ ARCHITECTURE.md          system diagrams & data flow
├─ FOLDER_STRUCTURE.md      what lives where, and why
├─ TECH_STACK.md            every dependency, justified
├─ DATABASE.md              tables, keys, relationships
├─ API_REFERENCE.md         every endpoint, fully specified
├─ AUTHENTICATION.md        identity & session lifecycle
├─ STATE_MANAGEMENT.md      client/server state ownership
├─ ROUTES.md                URL → view/controller map
├─ COMPONENTS.md            UI component contracts
├─ FEATURES/                one file per user-facing feature
│   ├─ login.md
│   ├─ dashboard.md
│   ├─ news.md
│   └─ admin.md
├─ DESIGN_DECISIONS.md      why this, not the alternative
├─ AI_CONTEXT.md             rules for future AI agents
├─ KNOWN_LIMITATIONS.md     accepted trade-offs, not bugs
├─ SECURITY.md               threat model & mitigations
├─ DEPLOYMENT.md             how a change reaches prod
├─ TESTING.md                what's covered, what isn't
├─ PERFORMANCE.md           known bottlenecks & budgets
└─ TROUBLESHOOTING.md        symptom → cause → fix
```

FIG. 1 – canonical docs/ tree, in read order for a first-time reader

READ ORDER FOR NEWCOMERS

README → PROJECT_OVERVIEW → ARCHITECTURE → the specific FEATURES/ file for the area they're touching → AI_CONTEXT before making any change.

How to Use This Playbook

This document is a **template and reasoning guide**, not a finished documentation set — it describes the structure, content standard, and diagrams that a real `docs/` folder should contain once it has been generated from an actual repository's source code.

IMPORTANT SCOPE NOTE

Producing the fully populated version of this system — real table names, real endpoints, real component props — requires analyzing an actual codebase. This playbook shows the target shape and quality bar for every file so that the generation step (done against your real repository) has a concrete standard to match.

Generation workflow

1. **Inventory** — walk the full repository tree; list every route, table, component, and environment variable before writing a word of prose.
2. **Draft structurally first** — create every file in the tree with headings only, so nothing is forgotten.
3. **Fill in "why" before "what"** — for each section, write the reasoning first; the mechanical description is the easy part.
4. **Diagram, then narrate** — build the Mermaid/architecture diagrams from actual imports and calls, not from memory of similar projects.
5. **Cross-link** — every FEATURES/ file should link to the API, DATABASE, and COMPONENTS sections it touches, and vice versa.
6. **Validate against the checklist** — see Part V, Section 20 — before considering a doc "done."

WHY STRUCTURAL-FIRST

Writing headings for all 26 files before filling any of them in surfaces gaps early — an empty FEATURES/payments.md heading is a visible TODO; a payments feature nobody thought to document is an invisible one.

PROJECT_OVERVIEW.md

docs/PROJECT_OVERVIEW.md

The front door for anyone — human or AI — who has never seen this project before. Written for a competent developer with zero context on *this specific* codebase, not zero context on software.

Required sections

Section	What it answers
What this project does	One paragraph, no jargon, as if explaining to a new hire on day one
Main modules	The 4–8 top-level pieces and the one-line job of each
How data flows	Where data originates, how it's transformed, where it's stored
Authentication process	Plain-language summary; full detail lives in AUTHENTICATION.md
User journey	The path a typical end user takes, screen by screen
Admin journey	What an operator/administrator can do differently
Third-party services	Every external API/service and what it's used for
Deployment process	One paragraph; full detail lives in DEPLOYMENT.md

QUALITY BAR

If a section could be copy-pasted into any other project's overview without changing a word, it's too generic — rewrite it using the project's actual module and table names.

ARCHITECTURE.md & Diagrams

docs/ARCHITECTURE.md

This is the file most worth getting right — it's the map everything else refers back to. Every diagram should be generated from the real dependency graph, not a generic three-tier sketch.

Application flow



FIG. 2 – request/response spine (replace with real layer names from the repo)

Authentication flow

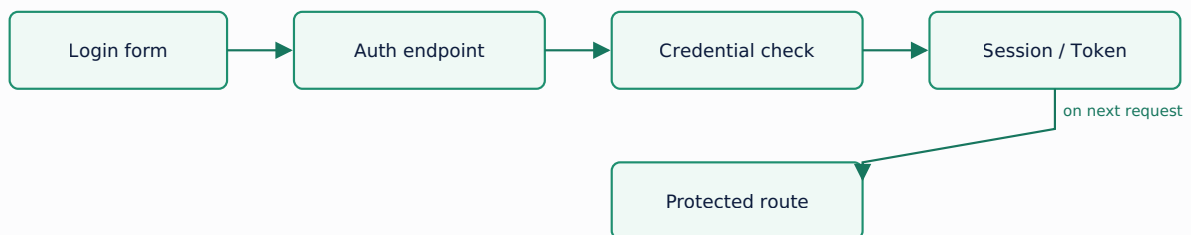


FIG. 3 – authentication lifecycle (full detail in AUTHENTICATION.md)

Diagrams to include

- ☐ Folder hierarchy (see FOLDER_STRUCTURE.md, Fig. 1)
- ☐ Application / request flow

- ☐ Authentication flow
- ☐ API request lifecycle (request → validation → handler → response)
- ☐ Database entity-relationship diagram
- ☐ Component hierarchy (top-level layout down to leaf components)
- ☐ User interaction flow (screen-to-screen)
- ☐ Admin interaction flow

WHY DIAGRAMS OVER PROSE HERE

Architecture is inherently spatial — "what calls what" is a graph, and graphs read faster as pictures. Prose descriptions of call graphs are where documentation and reality diverge fastest, because nobody re-reads a paragraph to check it's still accurate after a refactor; a diagram generated from real imports is easier to spot-check.

FOLDER_STRUCTURE.md & TECH_STACK.md

FOLDER_STRUCTURE.md

Not a directory listing — a directory listing *with intent*. Every top-level folder gets a sentence on what belongs there and, more importantly, what does **not** belong there.

Folder	Purpose	Does not contain
/src/features/*	Feature-scoped logic, colocated with its UI	Cross-cutting utilities
/src/shared/*	Reused primitives (buttons, hooks, formatters)	Anything with a single caller
/src/lib/*	Thin wrappers around third-party SDKs	Business logic

TECH_STACK.md

Every dependency, justified — not just versioned. This is where the "we chose X" reasoning lives so it never has to be re-litigated.

Layer	Technology	Why chosen
Frontend	(project's framework)	One line: fit for the team's skill set, ecosystem, or constraint that drove it
Backend	(project's runtime/framework)	
Database	(project's DB)	
Auth	(library/service)	
Hosting	(provider)	

TIP

Link each row to the fuller discussion in DESIGN_DECISIONS.md rather than duplicating the reasoning here.

DATABASE.md

docs/DATABASE.md

Per-table documentation standard

Field	What to record
Purpose	Why the table exists, in one sentence
Primary key	Column(s) and generation strategy (UUID, auto-increment, composite)
Foreign keys	Target table, on-delete behavior, and why that behavior was chosen
Indexes	Which columns, and the query pattern that requires them
Constraints	Uniqueness, check constraints, not-null rules and their business meaning
Column purposes	Every non-obvious column gets one line — especially booleans and enums

Entity relationship diagram

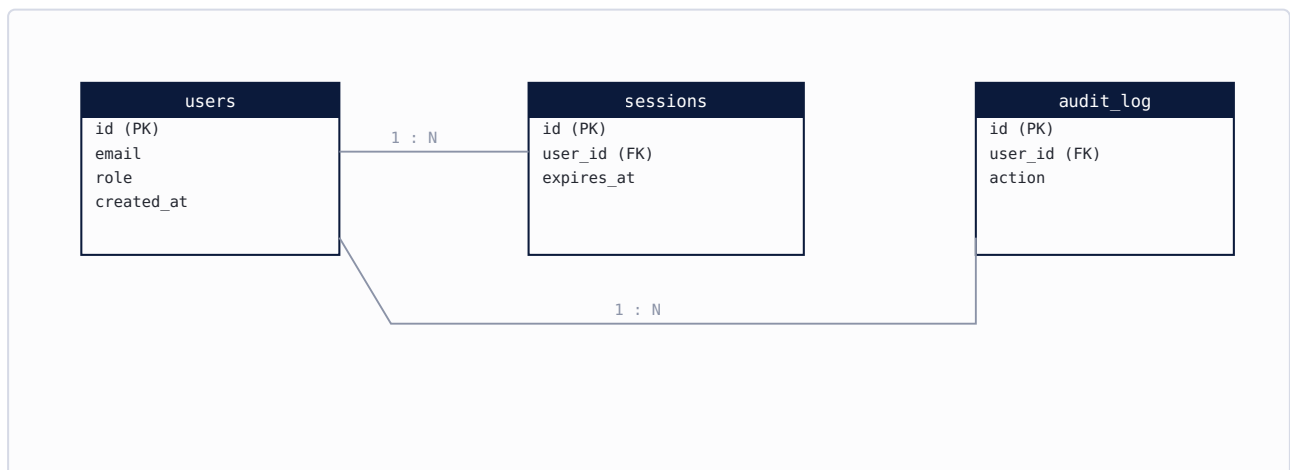


FIG. 4 – ER diagram template (replace tables/columns with the real schema)

API_REFERENCE.md

docs/API_REFERENCE.md

Every endpoint documented to the standard below — enough that a developer could implement a client without reading the handler code.

REQUEST LIFECYCLE

Document the pipeline every request passes through once, at the top of this file, then reference it — don't repeat "goes through the auth middleware" 40 times.

Worked example entry

Field	Value
Route	/api/v1/users/:id
Method	GET
Auth required	Yes — bearer token, owner or admin role
Path params	id — user UUID
Success response	200 — user object (see schema)
Error responses	401 not authenticated · 403 not owner/admin · 404 not found
Validation	Path param must be valid UUID format

Endpoint index format

Method	Route	Auth	Purpose
GET	/api/v1/users	Admin	List users
POST	/api/v1/users	Admin	Create user
PUT	/api/v1/users/:id	Owner/Admin	Update user
DELETE	/api/v1/users/:id	Admin	Remove user

API request lifecycle



FIG. 5 – the pipeline every endpoint entry can reference by name

AUTHENTICATION.md

docs/AUTHENTICATION.md

What to document

- ☐ Identity provider(s) and why they were chosen
- ☐ Session vs. token model, and expiry/refresh behavior
- ☐ Password storage — hashing algorithm and why
- ☐ Role/permission model and where it's enforced (middleware vs. handler vs. UI)
- ☐ Password reset and email-verification flows
- ☐ Rate limiting and lockout behavior on failed attempts

HIGH-RISK FOR AI AGENTS

Authentication code is disproportionately likely to contain intentional-looking "quirks" that are actually security controls. Flag this file's contents in AI_CONTEXT.md as requiring human review before modification.

STATE_MANAGEMENT.md & ROUTES.md

STATE_MANAGEMENT.md

Question	Answer it here
Where does server state live?	Cache layer, invalidation strategy
Where does client state live?	Global store vs. local component state, and the rule for choosing
What triggers a re-fetch?	Explicit list of invalidation events
What must never go in global state?	Secrets, large blobs, anything derivable from other state

ROUTES.md

Route	View / Controller	Auth	Notes
/	HomeView	Public	
/dashboard	DashboardView	User	Redirects to /login if unauthenticated
/admin/*	AdminLayout	Admin	Nested route guard, see AUTHENTICATION.md

COMPONENTS.md

docs/COMPONENTS.md

Per-component documentation standard

Field	What to record
Purpose	One sentence — what problem this component solves
Props	Name, type, required/optional, default
Internal state	What it tracks and why it isn't lifted up
Parent / child components	Where it's mounted, what it renders
Side effects	API calls, subscriptions, timers it owns
Rendering logic	Conditional branches worth calling out

Component hierarchy

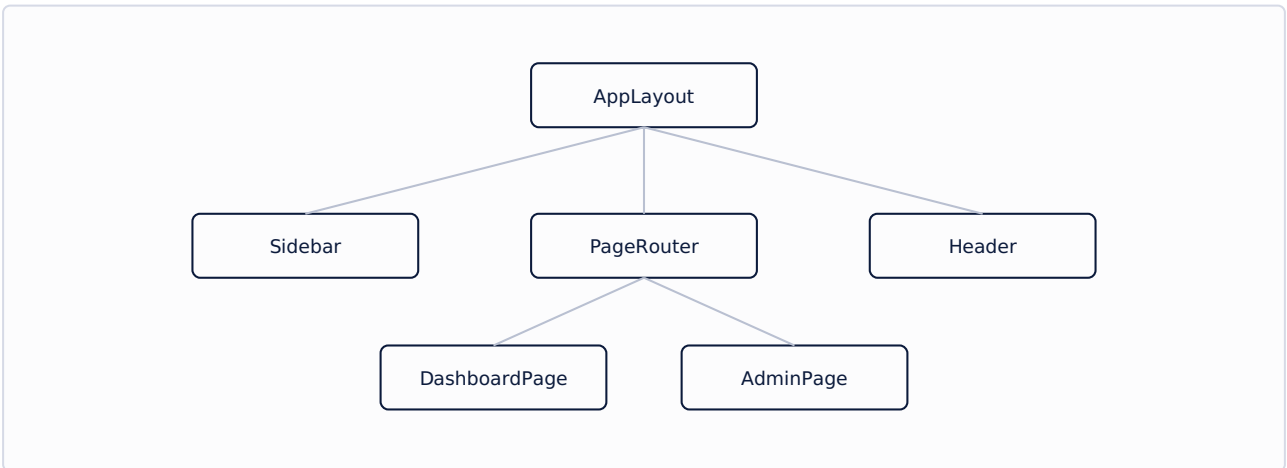


FIG. 6 – component tree template, top layout down to leaf pages

FEATURES/ — Per-Feature Template

`docs/FEATURES/feature-name.md`

One file per user-facing feature. This is where "why" gets the most space — features encode the most business reasoning of any layer in the system.

Required sections, in order

1. **Purpose** — the problem this feature solves, for whom
2. **How it works internally** — the mechanism, in prose, one level above the code
3. **Files & components involved** — exact paths, not descriptions
4. **APIs used** — links to the relevant `API_REFERENCE.md` entries
5. **Database tables involved** — links to `DATABASE.md` entries
6. **User flow** — step by step, from the end user's perspective
7. **Admin flow** — what changes for an operator, if anything
8. **Business rules & validation** — the rules a rewrite must preserve
9. **Edge cases handled** — the ones that would break silently if removed
10. **Error handling** — what the user sees when it fails, and why
11. **Security considerations** — anything feature-specific beyond `SECURITY.md`
12. **Performance considerations** — known costs, caching, N+1 risks
13. **Future improvements** — deferred work, and why it was deferred

WHY "BUSINESS RULES" GETS ITS OWN SECTION

Validation rules are the part of a feature most likely to be silently dropped during a rewrite, because they often live scattered across a form component, an API handler, and a database constraint — nowhere are they visible all at once except here.

Worked Example — login.md

docs/FEATURES/login.md

1. Purpose

Lets a returning user re-enter an authenticated session without re-registering. Solves: persistent identity across visits, without storing credentials client-side.

2. How it works internally

Credentials submitted to the auth endpoint are checked against a hashed password; on success, a signed session token is issued and set as an HTTP-only cookie. Subsequent requests carry the cookie and are validated by session middleware before reaching any protected route.

3. Files & components involved

- `src/features/auth/LoginForm`
- `src/features/auth/useAuth.hook`
- `src/api/auth/login.handler`
- `src/middleware/session.middleware`

4-5. APIs & tables

- `POST /api/v1/auth/login` — see `API_REFERENCE.md`
- `users` , `sessions` — see `DATABASE.md`

6. User flow

1. User submits email + password
2. Client-side validation checks format before submit
3. Server verifies credentials, issues session cookie
4. Client redirects to the last-attempted route, or dashboard

9. Edge cases handled

- ☐ Account exists but is unverified — distinct error, not "invalid credentials"
- ☐ Five consecutive failures triggers a temporary logout
- ☐ Session cookie present but expired — silent redirect to login, not an error page

DO NOT SIMPLIFY

The identical "invalid credentials" message for both wrong-password and unknown-email cases is deliberate — it prevents account enumeration. An AI agent "improving" error specificity here would reintroduce a known vulnerability.

DESIGN_DECISIONS.md

docs/DESIGN_DECISIONS.md

A running log of consequential choices, written in a consistent format so it can be scanned quickly and extended over time.

Entry format

Field	Content
Decision	What was chosen, in one line
Alternatives considered	What else was on the table, and why each was rejected
Trade-offs accepted	What this choice costs, explicitly
Reversibility	How hard this would be to undo later

TIP

Log the decision when it's made, not months later from memory — reasoning reconstructed after the fact tends to be a justification for the outcome rather than the actual trade-off analysis.

AI_CONTEXT.md

docs/AI_CONTEXT.md

WHY THIS FILE IS DIFFERENT

Every other file in docs/ explains the system to a reader. This one instructs an actor. It should read like an onboarding brief for a new engineer's first day, with explicit permission boundaries — not a general description.

Required contents

Section	Purpose
Coding & naming conventions	So new code is indistinguishable in style from old code
Folder conventions	Where new files of each type belong, by rule not by example
Reusable utilities	What already exists, to prevent re-implementation
Preferred architecture patterns	The pattern to extend, when more than one exists in the codebase
Things an AI should never change	Explicit list — see risk callout below
Safe areas to modify	Where experimentation carries low blast radius
High-risk files	Where a small change has large, non-obvious consequences
Project assumptions	Constraints treated as fixed (e.g. single-region, single-tenant)

"NEVER CHANGE" LIST — EXAMPLE ENTRIES

Session-cookie flags in the auth middleware · the password hashing algorithm/cost factor · migration files already applied to production · the audit-log write path.

SECURITY.md & KNOWN_LIMITATIONS.md

SECURITY.md

- ☐ Threat model — what this system explicitly defends against
- ☐ Input validation & sanitization strategy, and where it's enforced
- ☐ Secrets management — where they live, how they're rotated
- ☐ Dependency/audit policy
- ☐ Known accepted risks, with the reasoning for accepting them

KNOWN_LIMITATIONS.md

The difference between a limitation and a bug is that a limitation was a conscious trade-off. Recording it here prevents it from being "fixed" by accident and prevents it from being reported as a new bug.

Limitation	Why it exists	What would be needed to remove it
No offline support	Out of scope for v1 — real-time sync was prioritized	Service worker + conflict resolution strategy

DEPLOYMENT.md & TESTING.md

DEPLOYMENT.md

- ☐ Environments (dev / staging / prod) and how they differ
- ☐ Build & release pipeline, step by step
- ☐ Required environment variables and where they're sourced
- ☐ Rollback procedure
- ☐ Manual steps that aren't automated yet, and why

TESTING.md

Layer	What's covered	What's intentionally not
Unit	Pure functions, validation logic	
Integration	API handlers against a test database	
End-to-end	Critical user paths (login, checkout)	Admin-only flows, low traffic risk

PERFORMANCE.md & TROUBLESHOOTING.md

PERFORMANCE.md

- ☐ Known bottlenecks and the query/render path responsible
- ☐ Caching layers and their invalidation rules
- ☐ Performance budgets, if any (e.g. p95 response time targets)

TROUBLESHOOTING.md — entry format

Symptom	Likely cause	Fix	Logs to check
Login silently fails	Expired session cookie not cleared client-side	Clear cookie, re-auth	auth.log
Slow dashboard load	N+1 query on related records	Use the batched query helper	query.log

Also list the files that are **most frequently modified** when debugging — they're often the same handful of hotspots, and naming them saves the next investigator a search.

CHANGELOG.md & FUTURE_IMPROVEMENTS.md

CHANGELOG.md

Dated, human-readable entries — what changed and why, not a raw commit dump. Link back to the relevant DESIGN_DECISIONS.md entry when a change reflects a new decision.

FUTURE_IMPROVEMENTS.md

Improvement	Why it's not done yet	Prerequisite
Multi-region deployment	Not yet needed at current scale	Data residency decision

Documentation Quality Checklist

Run every file in docs/ against this list before considering the system complete.

- ☐ No placeholder text remains — every section reflects the real repository
- ☐ Every claim is based on code that was actually read, not assumed
- ☐ Every "what" is paired with a "why"
- ☐ Every diagram matches the current call graph / schema
- ☐ Every FEATURES/ file cross-links its API, database, and component references
- ☐ AI_CONTEXT.md's "never change" list has been reviewed by a human, not just generated
- ☐ KNOWN_LIMITATIONS.md distinguishes trade-offs from open bugs
- ☐ A reader with zero prior context could follow README → PROJECT_OVERVIEW → ARCHITECTURE and understand the system's shape

Documentation System Playbook — generated as a structural template. Populate each section against your actual repository's source before treating it as your project's source of truth.